

FIIT STU

Herné Anticheaty a podvody v hrách

Princípy informačnej bezpečnosti v hernom priemysle

Dominik Jurkas
2.10.2024

Obsah

Úvod	3
Ako fungujú anticheaty a aké existujú	4
Definícia anticheatov	4
Mechanizmus fungovania anticheatov	4
Typy anticheatov	5
Techniky obchádzania anticheatov	5
Výhody a nevýhody anticheatov	6
Výhody anticheatov:	6
Nevýhody anticheatov:	6
Ako hry reagujú na podvody	7
Okamžité opatrenia	7
Dlhodobé opatrenia	7
Varovania a upozornenia	7
Dopad na hernú komunitu	8
Začiatok implementácie Python skriptov. Návrh a výber vhodných knižníc	9
Návrh implementácie pre nedetekovateľný cheat (Triggerbot)	9
Návrh implementácie pre detekovateľný cheat (Wallhack)	9
Porovnanie a výber techník	10
Externé cheaty a ich prepojenie na hardvér	11
Hardvérové zariadenia používané na cheating	11
Technické základy fungovania	11
Interakcia s anticheat systémami	12
Výzvy pri detekcii a ochrana	12
Cheat Engine a jeho výzvy pre Anticheaty	13
Ako Cheat Engine Funguje	13
Techniky Používané Cheat Enginom	13
Nástrahy pre Anticheat Systémy	14
Metódy detekcie a blokovania Cheat Engine	14
Implementácia Python skriptov pre herné cheaty	15
Implementácia nedetekovateľného triggerbotu	15
1. Online verzia Counter-Strike 1.6	15

2. Valorant:	16
3. Deadlock.....	16
Implementácia detekovateľného wallhacku	20
Pokus o aktiváciu wallhacku v hre Deadlock (s VAC).....	20
Pokus o aktiváciu wallhacku v hre Rust (s EAC)	20
Dôvody, prečo to nefungovalo	21
Praktické použitie CheatEnginu	23
Záver	24
Referencie	25

Úvod

Herný priemysel patrí medzi najrýchlejšie rastúce odvetvia zábavného priemyslu, pričom digitálne hry priťahujú milióny hráčov po celom svete. S týmto nárastom sa zvyšujú aj požiadavky na udržiavanie spravodlivosti a integrity herného prostredia, čo je nevyhnutné pre zabezpečenie pozitívnych herných zážitkov pre všetkých účastníkov. Podvody v hrách, často označované ako "cheating", predstavujú závažný problém, ktorý nielenže narúša férovú súťaž medzi hráčmi, ale aj podkopáva dôveru v herné spoločnosti a ich produkty. Tento fenomén, ktorý zahŕňa využívanie softvérových a hardvérových nástrojov na získanie nespravodlivej výhody nad ostatnými hráčmi, môže mať ďalekosiahle následky pre celú hernú komunitu.

S cieľom predísť takýmto negatívnym vplyvom sa vyvinuli rôzne anticheat systémy, ktorých primárnou úlohou je aktívne monitorovanie herných procesov, odhaľovanie podozrivých aktivít a zabránenie akémukoľvek narušeniu pravidiel. Tieto systémy sa stali neoddeliteľnou súčasťou moderných online hier, kde je potrebné zabezpečiť, aby všetci hráči súťažili za rovnakých podmienok. Anticheat technológie sa neustále vyvíjajú a zdokonaľujú, aby dokázali držať krok s čoraz sofistikovanejšími podvodnými praktikami a zabezpečili tak dlhodobú stabilitu herného ekosystému.

Pri týchto výzvach treba pochopiť nielen základné princípy fungovania anticheat systémov, ale aj techniky, ktoré podvodníci používajú na obchádzanie týchto opatrení. Tento projekt sa bude zaoberať podrobnou analýzou anticheat riešení a rôznych foriem herných podvodov, aby poskytol komplexný pohľad na túto dôležitú tému v kontexte informačnej bezpečnosti v hernom priemysle.

Ako fungujú anticheaty a aké existujú

Anticheaty sú neoddeliteľnou súčasťou moderných online hier, zabezpečujú férovosť a integritu herného prostredia. Tieto systémy sú navrhnuté tak, aby aktívne monitorovali herné procesy, odhaľovali podozrivé aktivity a zabránili akémukoľvek narušeniu pravidiel. V tejto časti sa pozrieme na základné princípy fungovania anticheat systémov, rôzne typy anticheatov, techniky, ktoré podvodníci používajú na ich obchádzanie, a výhody a nevýhody jednotlivých prístupov.

Definícia anticheatov

Anticheat je softvér alebo systém navrhnutý na detekciu a prevenciu podvádžania v online hrách. Jeho hlavnou úlohou je zabezpečiť férovú hru tým, že identifikuje a blokuje hráčov, ktorí používajú nelegálne nástroje alebo modifikácie na získanie nespravodlivej výhody.

Podvádzanie v hrách môže zahŕňať rôzne aktivity, ako napríklad používanie aimbotov (automatické mierenie), wallhackov (videnie cez steny), skriptov na automatizáciu akcií alebo iných hackov, ktoré poskytujú hráčovi nespravodlivú výhodu nad ostatnými. Anticheat systémy pracujú na rôznych úrovniach, od detekcie podozrivého správania až po analýzu súborov a procesov bežiacich na hráčovom počítači.

Tieto systémy môžu byť implementované priamo do hry alebo môžu fungovať ako samostatné aplikácie, ktoré monitorujú a analyzujú herné prostredie.

Mechanizmus fungovania anticheatov

Anticheat systémy fungujú na detekcii a prevencii podvádžania v online hrách prostredníctvom rôznych mechanizmov. Tu sú niektoré z hlavných spôsobov, ako tieto systémy pracujú:

1. **Detekcia podozrivého správania:** Anticheat systémy monitorujú správanie hráčov v hre a hľadajú neobvyklé vzory, ktoré by mohli naznačovať podvádžanie. Napríklad, ak hráč vykazuje nadmerne presné mierenie alebo nereálne rýchle reakcie, systém môže označiť toto správanie ako podozrivé.
2. **Analýza súborov a procesov:** Anticheat softvér často kontroluje súbory a procesy bežiace na hráčovom počítači, aby zistil prítomnosť nelegálnych nástrojov alebo modifikácií. Ak systém nájde podozrivé súbory alebo procesy, môže hráča zablokovať alebo nahlásiť administrátorom.
3. **Overovanie integrity hry:** Niektoré anticheat systémy pravidelne kontrolujú integritu herných súborov, aby sa uistili, že neboli zmenené alebo manipulované. Ak systém zistí zmeny, môže hráča odpojiť od hry alebo mu zakázať prístup.

4. **Hybridné anticheat mechanizmy:** Moderné anticheat systémy používajú kombináciu rôznych techník na detekciu a prevenciu podvádzania. Tieto systémy môžu zahŕňať analýzu správania, kontrolu súborov a procesov, ako aj ďalšie pokročilé metódy.

Tieto mechanizmy spolupracujú na zabezpečení férovej hry a ochrane herného prostredia pred podvádzaním.

Typy anticheatov

Existuje niekoľko typov anticheatov, ktoré sa používajú na zabezpečenie férovosti v online hrách. Tu sú niektoré z najbežnejších:

1. **Klientské anticheaty:** Tieto programy bežia na počítači hráča a monitorujú jeho aktivitu, aby detekovali podvádzanie. Príklady zahŕňajú Easy Anti-Cheat (EAC) a BattlEye.
2. **Serverové anticheaty:** Tieto systémy bežia na herných serveroch a analyzujú dáta prichádzajúce od hráčov, aby identifikovali podozrivú aktivitu. Príkladom je Valve Anti-Cheat (VAC).
3. **Hybridné anticheaty:** Kombinujú prvky klientských a serverových anticheatov, aby poskytli komplexnejšiu ochranu. Príkladom je Riot Vanguard používaný v hre Valorant.
4. **Behaviorálne anticheaty:** Tieto systémy sledujú správanie hráčov v hre a hľadajú anomálie, ktoré by mohli naznačovať podvádzanie. Používajú pokročilé algoritmy a strojové učenie na detekciu podvodov.
5. **Hardvérové anticheaty:** Tieto riešenia zahŕňajú špeciálny hardvér, ktorý monitoruje a kontroluje činnosť počítača, aby zabránil podvádzaniu.

Techniky obchádzania anticheatov

Techniky obchádzania anticheatov sú neetické a často porušujú pravidlá a podmienky používania hier. Tieto techniky môžu zahŕňať:

1. **Modifikácia herných súborov:** Úprava alebo nahradenie súborov hry, aby sa zmenilo správanie hry alebo anticheatu.
2. **Injekcia kódu:** Vkladanie vlastného kódu do pamäte hry, aby sa obchádzali bezpečnostné mechanizmy.
3. **Používanie skriptov a makier:** Automatizácia herných akcií pomocou skriptov alebo makier, ktoré môžu byť ťažko detekovateľné.
4. **Obfuskácia:** Skrývanie alebo zamaskovanie podvodného softvéru, aby ho anticheat systémy nedokázali identifikovať.

5. **Virtuálne stroje a emulátory:** Spúšťanie hry v prostredí, ktoré je izolované od hlavného systému, čím sa sťažuje detekcia podvodov.

Výhody a nevýhody anticheatov

Anticheaty majú svoje výhody aj nevýhody, ktoré ovplyvňujú herný zážitok a bezpečnosť. Tu sú niektoré z hlavných:

Výhody anticheatov:

1. **Zlepšenie férovosti:** Anticheaty pomáhajú udržiavať rovnaké podmienky pre všetkých hráčov, čím zvyšujú férovosť a konkurencieschopnosť.
2. **Ochrana hernej ekonomiky:** V hrách, kde sa obchoduje s virtuálnymi predmetmi, anticheaty zabráňujú podvodom, ktoré by mohli narušiť ekonomiku hry.
3. **Zlepšenie herného zážitku:** Eliminácia podvodníkov zlepšuje celkový zážitok z hry pre poctivých hráčov.
4. **Dlhodobá udržateľnosť hry:** Hry s efektívnymi anticheatmi majú tendenciu udržať si hráčov dlhšie, pretože hráči majú dôveru v integritu hry.

Nevýhody anticheatov:

1. **Falošné pozitíva:** Niekedy môžu anticheaty nesprávne identifikovať poctivých hráčov ako podvodníkov, čo vedie k nespravodlivým zákazom.
2. **Výkonové problémy:** Niektoré anticheaty môžu zaťažovať systémové zdroje, čo môže viesť k zníženiu výkonu hry.
3. **Súkromie a bezpečnosť:** Klientské anticheaty často vyžadujú prístup k citlivým údajom na počítači hráča, čo môže vyvolávať obavy o súkromie a bezpečnosť.
4. **Obchádzanie:** Podvodníci neustále hľadajú nové spôsoby, ako obísť anticheaty, čo znamená, že vývojári musia neustále aktualizovať a zlepšovať svoje systémy.

Ako hry reagujú na podvody

Keď anticheat systém detekuje podvodníka, herné spoločnosti musia okamžite reagovať, aby zabezpečili férovosť a integritu hry. Tieto reakcie môžu zahŕňať rôzne formy zákazov a sankcií, ktoré majú za cieľ odradiť podvodníkov a ochrániť hernú komunitu.

Okamžité opatrenia

- **Vykopnutie z hry:** Hráč môže byť okamžite odstránený z aktuálnej hry alebo zápasu. Toto opatrenie je často používané v online multiplayer hrách, aby sa zabezpečila férovosť pre ostatných hráčov. Napríklad, v hrách ako “Counter-Strike: Global Offensive” alebo “Valorant” sú podvodníci okamžite vykopnutí z hry, aby sa minimalizoval ich vplyv na ostatných hráčov.
- **Dočasný zákaz:** Hráč môže dostať dočasný zákaz prístupu k hre alebo jej online službám. Tento zákaz môže trvať niekoľko hodín, dní alebo týždňov, v závislosti od závažnosti podvodu. V hrách ako “Overwatch” alebo “Rainbow Six Siege” sú dočasné zákazy bežné pre prvé priestupky.

Dlhodobé opatrenia

- **Trvalý zákaz:** V prípade opakovaných alebo závažných podvodov môže byť hráčovi udelený trvalý zákaz prístupu k hre. Tento zákaz môže zahŕňať aj zablokovanie účtu a stratu všetkých herných údajov. Hry ako “Fortnite” a “PUBG” sú známe tým, že udeľujú trvalé zákazy pre podvodníkov, aby udržali herné prostredie čisté.
- **Resetovanie účtu:** Niektoré hry môžu resetovať hráčov účet, čo znamená, že hráč stratí všetok pokrok, ktorý dosiahol. Toto opatrenie je menej bežné, ale môže byť použité v prípadoch, keď je podvod menej závažný, ale stále vyžaduje trest.
- **Hardware ban:** V niektorých prípadoch môžu herné spoločnosti pristúpiť k tzv. hardware banu, čo znamená, že konkrétne zariadenie, z ktorého bol podvod spáchaný, bude zablokovaný. Toto opatrenie je veľmi účinné, pretože podvodník nemôže jednoducho vytvoriť nový účet na tom istom zariadení.

Varovania a upozornenia

Niektoré hry môžu najprv poslať varovanie, aby hráča upozornili na nelegitímne aktivity. Toto varovanie môže byť sprevádzané dočasným zákazom alebo inými sankciami. Napríklad, v hre “League of Legends” môžu hráči dostať varovanie pred tým, ako dostanú dočasný zákaz.

Dopad na hernú komunitu

Tieto opatrenia majú významný dopad na hernú komunitu. Na jednej strane pomáhajú udržiavať férové a zábavné prostredie, čo je kľúčové pre dlhodobú spokojnosť hráčov. Na druhej strane môžu prísne tresty odradiť nových hráčov alebo spôsobiť frustráciu u tých, ktorí boli nespravodlivo obvinení. Preto je dôležité, aby anticheat systémy boli čo najpresnejšie a spravodlivé.

Začiatok implementácie Python skriptov. Návrh a výber vhodných knižníc

Pri implementácii skriptov pre detekovateľné a nedetekovateľné cheaty je nevyhnutné zvoliť vhodné knižnice a nástroje, ktoré umožnia efektívnu a flexibilnú prácu s herným prostredím bez nutnosti priamo zasahovať do pamäte hry. V tejto časti sa zameriame na výber knižníc, ktoré budú použité na vytvorenie nedetekovateľného triggerbotu a detekovateľného cheatu.

Návrh implementácie pre nedetekovateľný cheat (Triggerbot)

Pre nedetekovateľný cheat typu triggerbot sme sa rozhodli pre techniku detekcie zmeny farby pixelov na obrazovke. Tento prístup využíva vizuálne dáta hry, namiesto manipulácie s hernou pamäťou, čo minimalizuje riziko detekcie anticheat systémami.

Pre tento účel boli vybrané nasledujúce knižnice:

- Pillow: Knižnica na spracovanie obrázkov v Pythone, ktorá umožňuje získavanie farieb pixelov z obrazovky pomocou funkcie `ImageGrab.grab()`.
- PyAutoGUI: Knižnica na simuláciu používateľských akcií, ako sú kliknutia myši, stlačenie klávesov alebo pohyby myši. Táto knižnica je ideálna na implementáciu triggerbotu, kde bude kliknutie simulované na základe zmeny farby pixelu v zameriavacom križi.

Tento návrh umožňuje flexibilné a menej nápadné ovládanie bez zásahov do hernej pamäte, čo je základnou výhodou pre minimalizáciu detekcie.

Návrh implementácie pre detekovateľný cheat (Wallhack)

Pri výbere knižníc pre detekovateľný cheat typu wallhack, ktorý manipuluje s viditeľnosťou nepriateľských modelov, sme sa rozhodli pre prístup, ktorý priamo modifikuje hernú pamäť. Tento cheat umožní hráčovi vidieť nepriateľov aj cez steny, a to prostredníctvom úpravy hodnôt v pamäti hry.

Pre tento účel bola zvolená nasledujúca knižnica:

- PyMem: Knižnica, ktorá umožňuje pripojenie k externému procesu (hry) a manipuláciu s jeho pamäťou. Pomocou tejto knižnice budeme schopní čítať a modifikovať pamäťové adresy, ktoré ovplyvňujú viditeľnosť herných objektov (nepriatelia, steny).

Tento cheat bude vykonávať priame zásahy do pamäti hry, čím bude možné spraviť nepriateľov vždy viditeľnými, bez ohľadu na prekážky (napr. steny). Tento prístup je však vysoko rizikový, pretože anticheat systémy často sledujú manipuláciu s hernými štruktúrami, a teda je vysoká pravdepodobnosť detekcie.

Porovnanie a výber techník

Oba prístupy majú svoje výhody a nevýhody. Kým prvý skript využívajúci vizuálne dáta ponúka nízke riziko detekcie, jeho nevýhodou môže byť obmedzená presnosť a rýchlosť v závislosti na kvalite obrazu a vykreslenia hry. Druhý skript, založený na prístupe do pamäte hry pre wallhack je efektívnejší a priamo umožňuje manipulovať s viditeľnosťou nepriateľov, ale je podstatne náchylnejší na detekciu, čo môže viesť k permanentným zákazom v hre.

Externé cheaty a ich prepojenie na hardvér

Podvádzanie v hrách často zahŕňa softvérové úpravy, ale existuje aj sofistikovanejšia forma cheatovania, ktorá využíva hardvérové zariadenia. Tieto externé cheaty sú navrhnuté tak, aby obchádzali bežné softvérové anticheat systémy, čím sa stávajú omnoho náročnejšie na detekciu. V tejto kapitole sa pozrieme na princípy fungovania takýchto zariadení, ich interakciu s hernými dátami a dôvody, prečo sú tieto metódy ťažšie odhaliteľné.

Hardvérové zariadenia používané na cheating

Medzi najčastejšie používané hardvérové zariadenia na cheating patria:

- **PCIe karty:** Tieto karty sa zasúvajú do PCIe slotu základnej dosky a umožňujú prístup k dátam prenášaným v rámci systému. Takéto zariadenia môžu sledovať a analyzovať dáta v reálnom čase a poskytovať informácie, ktoré sa dajú využiť na zlepšenie herného výkonu, napríklad pri automatickom mierení, alebo vidieť cez steny.
- **USB moduly:** Zariadenia pripojené cez USB port môžu obsahovať špeciálne naprogramované mikrokontroléry, ktoré simulujú klávesnicu a myš, prípadne môžu prenášať príkazy s vysokou presnosťou. Tieto zariadenia umožňujú hráčovi vykonávať skripty na automatizáciu akcií, ako sú rýchle reakcie alebo presné strely, pričom sú pre softvérové anticheaty ťažko odhaliteľné.

Technické základy fungovania

Hardvérové zariadenia, ktoré sa používajú na cheating, fungujú nezávisle od softvérových procesov na počítači. Ich princíp spočíva v monitorovaní alebo modifikovaní údajov bez priameho zásahu do pamäte hry. Týmto spôsobom sa dokážu vyhnúť bežným detekčným metódam, ktoré sledujú softvérové anomálie. Napríklad:

- **PCIe karty** môžu byť naprogramované tak, aby zachytávali signály z grafickej karty a analyzovali vizuálne údaje. Tieto údaje sa spracujú mimo hlavného procesora a operačného systému, čo znemožňuje ich detekciu bežnými softvérovými anticheat systémami.
- **USB moduly** môžu simulovať vstupy používateľa a automatizovať pohyby myši a klávesové vstupy. Takéto skripty sú často považované za „legitímne“ používateľské akcie, čo sťažuje ich odhalenie.

Interakcia s anticheat systémami

Anticheat systémy sú navrhnuté tak, aby detekovali softvérové anomálie a zmeny v hernej pamäti. Pri detekcii externých hardvérových podvodov sa však stretávajú s množstvom obmedzení:

- **Obmedzený prístup k hardvérovým údajom:** Väčšina anticheatov nemá prístup k nízkoúrovňovým hardvérovým informáciám, ktoré sú potrebné na odhalenie PCIe kariet alebo USB modulov.
- **Nezávislosť od softvérového prostredia:** Externé zariadenia operujú mimo bežných softvérových obvodov, čo znamená, že sú pre anticheaty prakticky neviditeľné. Napríklad, PCIe karta môže analyzovať grafické výstupy a poskytovať spätnú väzbu bez zásahu do bežiaceho herného procesu.

Výzvy pri detekcii a ochrana

Detekcia hardvérových podvodov je pre vývojárov hier a anticheat riešení mimoriadne náročná. Existujú však určité postupy a technológie, ktoré sa začínajú používať na boj proti týmto podvodom:

- **Sledovanie neštandardných zariadení:** Niektoré pokročilé anticheat systémy (kernel anticheaty) sa snažia monitorovať pripojené zariadenia a zisťovať ich správanie. To zahŕňa overenie, či sú pripojené USB zariadenia alebo podozrivé moduly.
- **Analýza vstupov:** Systémy môžu analyzovať časové vzory vstupov (napr. rýchlosť a konzistentnosť pohybov myši) a detegovať neobvyklé správanie, ktoré môže naznačovať použitie automatizácie.

Tieto techniky však vyžadujú vysokú úroveň sofistikovanosti a môžu byť obmedzené právnymi a etickými otázkami, pretože niektoré z nich môžu zasahovať do súkromia hráčov.

Cheat Engine a jeho výzvy pre Anticheaty

Cheat Engine je jedným z najznámejších a najrozšírenejších nástrojov využívaných hráčmi na manipuláciu s hernými premennými. Je to open-source aplikácia, ktorá umožňuje užívateľom pristupovať k pamäti behu hry, čo dáva možnosť hľadať a modifikovať hodnoty priamo v procese hry. Tieto zmeny môžu zahŕňať úpravy, ako je zvýšenie životov, nekonečná munícia, alebo úpravy herných mechaník v reálnom čase.

Ako Cheat Engine Funguje

Cheat Engine je založený na skenovaní a modifikácii pamäte. Táto technika umožňuje hráčovi prehľadávať pamäť hry pre konkrétne hodnoty a meniť ich podľa potreby. Postup je jednoduchý:

1. **Skenovanie Pamäte:** Hráč najprv zadá hľadanú hodnotu, napríklad počet životov. Cheat Engine následne prehľadá pamäť procesu hry a identifikuje všetky miesta, kde sa daná hodnota nachádza.
2. **Následné Filtrovanie:** Keď sa hodnota v hre zmení, napríklad hráč stratí život, Cheat Engine umožňuje opätovné skenovanie, čím sa zníži počet možných pamäťových adries na presnejší cieľ.
3. **Modifikácia Pamäte:** Po identifikácii správnej adresy hráč môže zmeniť hodnotu, napríklad zvýšiť počet životov na maximum.

Techniky Používané Cheat Enginom

Cheat Engine využíva rôzne techniky na prístup a modifikáciu pamäte:

- **Dynamické skenovanie:** Schopnosť identifikovať hodnoty aj pri meniacich sa pamäťových adresách, čo je spôsobené použitím pointerov a dynamického pridelovania pamäte v moderných hrách.
- **Kódy na zmenu:** Manipulácia s inštrukciami programu, ktorá umožňuje zmeniť logiku hry, napríklad zmeniť príkazy, ktoré kontrolujú počet životov alebo munície.
- **Injektovanie kódu:** Vkladanie vlastných inštrukcií do pamäte hry, čo umožňuje pridanie nových funkcií alebo manipuláciu hernej logiky podľa preferencií užívateľa.

Nástrahy pre Anticheat Systémy

Detekcia Cheat Engine predstavuje pre anticheat systémy výraznú výzvu z niekoľkých dôvodov:

- **Dynamický a flexibilný nástroj:** Cheat Engine sa dá prispôbiť tak, aby obchádzal základné ochranné mechanizmy hier. Mnoho používateľov vie tento nástroj modifikovať alebo zamaskovať jeho prítomnosť.
- **Nízka úroveň prístupu:** Cheat Engine môže pracovať na úrovni systému s povoleniami na čítanie a zápis pamäte, čo sťažuje detekciu jeho činnosti pomocou bežných ochranných opatrení v hrách.
- **Podpora pre obfuskáciu:** Pokročilejší užívatelia môžu nástroj maskovať a modifikovať jeho podpisy, čím sa vyhýbajú detekcii prostredníctvom anticheat systémov, ktoré sa spoliehajú na identifikáciu podľa známych súborov a vzorcov správania.

Metódy detekcie a blokovania Cheat Engine

Anticheat systémy využívajú rôzne techniky na detekciu a blokovanie Cheat Engine:

- **Skenovanie pamäťových prístupov:** Monitorovanie zmien v pamäti a detekcia neoprávneného prístupu pomocou systémových nástrojov.
- **Detekcia podpisov:** Hľadanie známych kódov a inštrukcií spojených s Cheat Enginom, ktoré môžu byť v pamäti alebo na disku.
- **Anti-tamper techniky:** Ochrana kódu hry pred modifikáciou a vkladom cudzieho kódu, často implementovaná pomocou technológií, ako je VMProtect alebo Themida.
- **Sandboxové prostredie:** Hry sa spúšťajú v izolovaných prostrediach, kde sú prístupy k pamäti obmedzené a sledované, čím sa znižuje riziko úspešného využitia Cheat Engine.

Cheat Engine predstavuje pre herné anticheaty významnú výzvu vďaka jeho pokročilým schopnostiam skenovať a modifikovať pamäť hry. Jeho detekcia je náročná kvôli možnosti maskovania a modifikácií zo strany používateľov. Anticheat systémy preto musia využívať sofistikované metódy na monitorovanie, detekciu a blokovanie nástrojov tohto typu, pričom sa neustále prispôbujú novým technikám, ktoré umožňujú hráčom obísť ochranu.

Implementácia Python skriptov pre herné cheaty

V tejto kapitole sa venujeme praktickej časti projektu, ktorá zahŕňa podrobnú implementáciu skriptov pre dva typy herných cheatov: nedetekovateľný triggerbot a detekovateľný wallhack. Popíšeme výber a použitie vhodných knižníc, vysvetlíme detaily implementácie a poskytneme kód s podrobnými vysvetleniami.

Implementácia nedetekovateľného triggerbotu

Triggerbot je typ cheatu, ktorý automaticky vykoná akciu (v našom prípade kliknutie) na základe detekcie určitej farby na obrazovke, čo znamená, že hráč nemusí manuálne vykonávať tieto akcie. Týmto spôsobom môže hráč získať výhodu nad ostatnými bez priameho zásahu do hernej pamäte, čo minimalizuje riziko detekcie anticheat systémami.

Ako sme sa už v minulej kapitole dozvedeli, na nedetekovaný cheat budeme používať tieto knižnice

- **Pillow:** Knižnica na spracovanie obrázkov a zachytávanie pixelov na obrazovke.
- **PyAutoGUI:** Knižnica umožňujúca simuláciu používateľských akcií, ako sú kliknutia myši a pohyby.

Taktiež použijeme aj tieto knižnice, ktoré slúžia na základné programovanie v pythone:

- **Time:** Knižnica na prácu s časovými jednotkami
- **Random:** Knižnica umožňujúca výberu náhodnej hodnoty z intervalu
- **Keyboard:** Knižnica použitá na sledovanie, či je stlačené aktivovacie tlačidlo
- **Ctypes:** Knižnica umožňujúca priamu simuláciu hardvérových vstupov

Testoval som program, ktorý som vytvoril na praktickú časť môjho projektu, na troch hrách: Valorant, online verzii Counter-Strike 1.6 a Deadlock. Každý z týchto testov mal svoje špecifiká a odhalil rôzne úrovne efektivity a bezpečnostných opatrení implementovaných anticheatmi.

1. Online verzia Counter-Strike 1.6: Táto hra je známa svojím starším anticheat systémom, ktorý sa zameriava najmä na sledovanie správania hráča. Tento anticheat nie je tak sofistikovaný ako moderné riešenia, čo mi umožnilo vytvoriť a otestovať triggerbot s relatívne jednoduchou úpravou časovania. V rámci testu som simuloval realistické reakčné časy, čo znamená, že triggerbot reagoval v takom čase, ktorý by bol podobný ľudskej reakcii. To pomohlo pri obídení detekcie anticheatu, ktorý analyzuje neprirodzene rýchle reakcie ako podozrivé.

Pri testovaní som však narazil na konkrétny problém s mapami, ktoré obsahujú dynamické efekty, ako napríklad snehové vločky. Keď snehová vločka prešla cez stred obrazovky, triggerbot reagoval spustením výstrelu, pretože zmena farby pixelu v centre obrazovky bola interpretovaná ako cieľ. Riešením by bolo implementovať podmienku, ktorá rozpozna farbu vločiek a pridá kontrolu na ignorovanie výstrelu, ak sa zmena zhoduje s týmto špecifickým efektom. Tento prístup by síce vyžadoval ďalšie úpravy a analýzu mapy, ale naznačuje možnosti, ako sa prispôbiť zmenám prostredia bez toho, aby bol triggerbot detekovaný.

2. Valorant: Táto hra predstavuje významnú výzvu pre testovanie, a to najmä kvôli svojmu pokročilému kernel-level anticheatu, známemu ako Vanguard. Tento anticheat monitoruje aktivity v operačnom systéme na nízkej úrovni a zabezpečuje, aby hráč nemohol jednoducho obísť bezpečnostné mechanizmy pomocou bežných softvérových knižníc. V prvom teste som využil knižnice ako keyboard a pyautogui, ktoré sú obľúbené na automatizáciu vstupov, avšak Vanguard okamžite tieto knižnice detekoval a zablokoval spustenie cheatu.

Po tomto zlyhaní som musel prepracovať celý kód a prejsť na menej detekovateľné riešenie. Použil som knižnicu ctypes, ktorá umožňuje simuláciu hardvérových vstupov, čo je metóda menej náchylná na odhalenie. Avšak aj napriek tejto zmene som narazil na problémy. Triggerbot hlásil zmenu pixelu, aj keď sa v skutočnosti nič nezmenilo na obrazovke, a navyše, samotné spúšťanie výstrelův nefungovalo správne. Tento výsledok mi ukázal, že Vanguard je schopný detekovať a obmedziť podobné softvérové manipulácie na úrovni, ktorá si vyžaduje sofistikovanejšie metódy na obídenie jeho detekčných systémův. Tieto metódy môžu zahŕňať napríklad použitie ovládačov alebo iných spôsobův, ktoré emulujú skutočné hardvérové zariadenia.

Výsledky testu potvrdili, že kernel-level anticheaty ako Vanguard sú veľmi účinné a že ich prekonanie jednoduchými skriptami je mimoriadne náročné. Tento typ anticheatu funguje na princípe monitorovania systémových volaní a prístupu k hardvérovým vstupom, čo značne sťažuje prácu podvodníkom. Zároveň zabezpečuje, že akákoľvek podozrivá aktivita na úrovni softvéru je okamžite zaznamenaná a hráčovi je obmedzený prístup k hre.

3. Deadlock: Táto hra predstavuje modernú hru s implementovaným serverovým anticheatom, konkrétne systémom Valve Anti-Cheat (VAC). Tento typ anticheatu pracuje na serverovej úrovni a analyzuje správanie hráča z hľadiska serverových vzorcův. Prvý pokus so starším kódom, ktorý využíval knižnicu pyautogui, neprešiel, pretože server detekoval neštandardné vstupy a blokoval ich. Avšak po úprave kódu a využití knižnice

ctypes som bol schopný dosiahnuť úspech – triggerbot fungoval bez problémov a dokázal som simulovať výstrely bez detekcie.

Je dôležité poznamenať, že serverové anticheaty, ako je VAC, sú účinné pri detekcii vzorcov správania, avšak ich schopnosť odhaliť simulácie vstupov závisí od rozsahu a pokročilosti, ktoré používajú. Keďže Deadlock je novšia hra, jej anticheat môže mať menej skúseností s rôznymi typmi cheatov, čo umožnilo, aby môj triggerbot úspešne fungoval. Avšak aj tento test som vykonával výhradne na offline verzii hry, pretože by bolo neetické a protizákonné testovať takýto softvér na reálnych hráčoch v online prostredí.

Celkové výsledky mojich testov ukázali, že efektivita anticheatov sa výrazne líši podľa toho, aké metódy a technológie využívajú. Kernel-level anticheaty sú najúčinnnejšie a vyžadujú mimoriadne sofistikované techniky na obídenie, zatiaľ čo serverové anticheaty sa spoliehajú na analýzu vzorcov a môžu byť obídené s dôkladným plánovaním a použitím menej známych techník.

Program s pomocou knižnice PYAUTOGUI:

```
from PIL import ImageGrab
import pyautogui
import time
import random
import keyboard
import tkinter as tk

# Konfiguračné parametre
root = tk.Tk()
screen_width = root.winfo_screenwidth()
screen_height = root.winfo_screenheight()
x, y = screen_width/2, screen_height/2 # Pozícia na stred obrazovky
activation_key = 'x' # Kláves, ktorého držanie aktivuje triggerbot
min_delay = 0.05 # Minimálny časový interval v sekundách
max_delay = 0.1 # Maximálny časový interval v sekundách
# Funkcia na získanie farby stredového pixelu
def get_center_pixel_color():
    screenshot = ImageGrab.grab()
    return screenshot.getpixel((x, y))
# Počiatočné získanie referenčnej farby
initial_color = get_center_pixel_color()
print("Triggerbot je pripravený. Držte 'x' na aktiváciu.")
while True: # Nekonečný cyklus na sledovanie farby a simuláciu kliknutia pri držaní klávesu
    if keyboard.is_pressed(activation_key):
        current_color = get_center_pixel_color()
        # Kontrola zmeny farby oproti počiatočnej
        if current_color != initial_color:
            # Generovanie náhodného časového oneskorenia
            delay = random.uniform(min_delay, max_delay)
            time.sleep(delay) # Čakanie pred simuláciou kliknutia
            pyautogui.leftClick()
            print(f"Výstreľ s oneskorením {delay:.2f} sekúnd")
            # Aktualizácia počiatočnej farby po výstrele
            initial_color = current_color
        time.sleep(0.01)
```

Program s pomocou ctypes:

```
import ctypes
import time
import random
import keyboard
import tkinter as tk
from PIL import ImageGrab

root = tk.Tk() # Konfiguračné parameter
screen_width = root.winfo_screenwidth()
screen_height = root.winfo_screenheight()
x, y = int(screen_width / 2), int(screen_height / 2)
activation_key = 'x'
min_delay = 0.05
max_delay = 0.1
MOUSEEVENTF_LEFTDOWN = 0x0002 # Konštanty pre kliknutie myši
MOUSEEVENTF_LEFTUP = 0x0004

def click(): # Funkcia pre simuláciu kliknutia myši pomocou WinAPI
    ctypes.windll.user32.mouse_event(MOUSEEVENTF_LEFTDOWN, 0, 0, 0, 0)
    time.sleep(0.01) # Krátke oneskorenie na simuláciu realistického kliknutia
    ctypes.windll.user32.mouse_event(MOUSEEVENTF_LEFTUP, 0, 0, 0, 0)

def get_center_pixel_color(): # Funkcia na získanie farby stredového pixelu
    screenshot = ImageGrab.grab()
    return screenshot.getpixel((x, y))

initial_color = get_center_pixel_color() # Počiatočné získanie referenčnej farby
print("Triggerbot je pripravený. Držte 'x' na aktiváciu.")

while True: # Nekonečný cyklus na sledovanie farby a simuláciu kliknutia pri držaní klávesu
    if keyboard.is_pressed(activation_key):
        current_color = get_center_pixel_color()
        if current_color != initial_color: # Kontrola zmeny farby oproti počiatočnej
            delay = random.uniform(min_delay, max_delay)
            time.sleep(delay)
            click()
            print(f"Výstrel s oneskorením {delay:.2f} sekúnd")
            initial_color = current_color # Aktualizácia počiatočnej farby po výstrele
        time.sleep(0.01)
```

Implementácia detekovateľného wallhacku

V tejto časti projektu som sa zamerlal na experimentovanie s detekovateľným wallhackom, konkrétne na manipuláciu s pamäťou hry s cieľom aktivovať wallhack, ktorý by umožnil vidieť cez steny. Moje testovanie bolo zamerané na to, aby som zistil, ako anticheat systémy, ako Valve Anticheat (VAC) a Easy Anti-Cheat (EAC), reagujú na takéto pokusy o manipuláciu. Na pokus som si vybral hry Deadlock (s VAC) a Rust (s EAC), kde som sa pokúsil testovať svoje experimenty v online prostredí. Experimenty prebiehali na súkromných serveroch, aby som neovplyvnil iných hráčov.

Pokus o aktiváciu wallhacku v hre Deadlock (s VAC)

Prvým krokom bolo identifikovať pamäťovú adresu, ktorá zodpovedá za aktiváciu wallhacku. Použil som nástroj Cheat Engine na skenovanie pamäte hry Deadlock, kde som sa pokúsil nájsť hodnotu, ktorá by ovplyvňovala viditeľnosť objektov cez steny. Bohužiaľ, hľadanie adresy nebolo úspešné a nakoniec som sa rozhodol použiť náhodne získanú adresu. Cieľom bolo upraviť hodnoty, aby som dokázal že sa dá meniť hodnota v hre, čiže ak sa nájde adresa pre wallhack, tak tiež bude možná zmeniť a tým úspešne aktivovať.

Použil som knižnicu Pymem v Pythone na zápis do pamäte hry s cieľom aktivovať wallhack. Kód bol nasledovný:

Pri spustení kódu sa prístup k pamäti okamžite zablokoval. Anticheat (VAC) detekoval pokus o manipuláciu a okamžite blokoval zápis do pamäti bez akéhokoľvek viditeľného efektu v hre. Neexistovala žiadna reakcia zo strany hry, ako napríklad ban alebo vyhodenie z hry. Každý pokus o manipuláciu s pamäťou bol jednoducho zamietnutý, bez toho, aby anticheat poskytol nejakú formu odozvy.

Pokus o aktiváciu wallhacku v hre Rust (s EAC)

Rovnaký postup som aplikoval aj na hru Rust, ktorá používa Easy Anti-Cheat (EAC). Po spustení hry a pokuse o manipuláciu s pamäťou (prípadne zápis hodnoty pre wallhack), bola reakcia zo strany EAC okamžitá. EAC detekoval pokus o manipuláciu s pamäťou a takisto blokoval prístup. Tento pokus o zápis do pamäte neprešiel, rovnako ako v prípade VAC, a nedostal som žiadnu vizuálnu odozvu zo strany hry.

EAC implementuje pokročilé techniky ochrany pamäte a monitoruje neautorizované pokusy o manipuláciu s pamäťou. Výsledkom bolo, že som nebol schopný úspešne zapísať do pamäte a získať žiadne vizuálne zmeny v hre.

Dôvody, prečo to nefungovalo

- **Ochrana pamäte anticheatom:** Anticheat systémy, ako VAC a EAC, používajú pokročilé techniky na ochranu pamäte, ktoré sledujú a blokujú neautorizované pokusy o zápis do kritických oblastí pamäte. Keď anticheat detekuje takýto pokus, okamžite blokuje prístup a vykoná opatrenia na zamedzenie úspešného podvodu.
- **Nepovolený zápis do pamäte:** Keď som sa pokúsil zapisovať hodnoty do pamäte hry, anticheat okamžite detekoval nezvyčajné správanie a zablokoval prístup, aby zabránil manipulácii.
- **Nedostatočná reakcia zo strany hry:** Keďže zápis do pamäte bol zablokovaný, nebolo možné dosiahnuť vizuálnu zmenu v hre, ktorá by naznačovala, že wallhack funguje. Bez úspešnej manipulácie s pamäťou neexistovala žiadna viditeľná zmena ani žiadne ďalšie logické reakcie zo strany hry.

Pri pokusoch o aktiváciu detekovateľného wallhacku v hrách Deadlock (s VAC) a Rust (s EAC) som narazil na okamžité blokovanie prístupu k pamäti zo strany anticheat systémov. Anticheat systémy, ako VAC a EAC, sú vysoko účinné pri detekcii a blokovaní pokusov o manipuláciu s pamäťou. V oboch prípadoch neexistovala žiadna vizuálna reakcia zo strany hry, ako napríklad ban alebo vyhodenie z hry, pretože pokusy o manipuláciu s pamäťou boli ihneď detegované a zablokované ešte predtým, než mohlo dôjsť k akémukoľvek viditeľnému efektu.

Kód ktorý som použil:

```
import pymem
def open_game_process(process_name): # Otvorí herný proces
    try:
        pm = pymem.Pymem(process_name)
        print(f"Herný proces {process_name} bol úspešne otvorený.")
        return pm
    except Exception as e:
        print(f"Chyba pri otváraní procesu: {e}")
        return None
def modify_memory_value(pm, address, new_value): # Pokusí sa zmeniť hodnotu na pamäťovej adrese
    try:
        pm.write_int(address, new_value)
        print(f"Hodnota na adrese {hex(address)} bola úspešne zmenená na {new_value}.")
    except Exception as e:
        print(f"Chyba pri zápise do pamäte na adrese {hex(address)}: {e}")
def main():
    process_name = input("Zadajte názov herného procesu (napr. 'valorant.exe'): ") # Zadanie názvu herného procesu
    pm = open_game_process(process_name) # Otvorí proces
    if pm is None:
        return
    try:
        memory_address = int(input("Zadajte pamäťovú adresu (hexadecimálne, napr. '0x17CA0'): "), 16) # Zadanie pamäťovej adresy
    except ValueError:
        print("Zadaná adresa je neplatná.")
        return
    try:
        new_value = int(input("Zadajte novú hodnotu (celé číslo): ")) # Zadanie novej hodnoty
    except ValueError:
        print("Neplatná hodnota.")
        return
    modify_memory_value(pm, memory_address, new_value) # Pokus o zmenu hodnoty na pamäťovej adrese
if __name__ == "__main__":
    main()
```

Praktické použitie CheatEngineu

V tejto časti projektu som sa zamerlal na praktické využitie nástroja Cheat Engine v offline prostredí hry Deadlock. Cieľom bolo manipulovať s hodnotami v pamäti hry, konkrétne s hodnotami, ktoré ovplyvňujú životy a muníciu, aby som dosiahol efekt, ako je nekonečná munícia alebo nekonečné životy. Tento experiment bol úspešný, pričom celkový proces som zaznamenal na video, ktoré ukazuje celý priebeh experimentu.

Postup experimentu:

1. **Identifikácia pamätevej adresy:** Prvým krokom bolo nájsť správnu adresu hry Deadlock v pamäti. Pomocou Cheat Engine som prehľadával pamäť, aby som našiel hodnotu, ktorá sa zhoduje s hodnotou života alebo munície. Začal som s vyhľadávaním aktuálnej hodnoty a postupne som overoval jej zmeny v priebehu hry.
2. **Prehľadávanie a filtrovanie hodnôt:** Po začiatku prehľadávania pamäte som zadal hodnotu, ktorá zodpovedala aktuálnemu množstvu munície alebo života. Následne som zmenil tieto hodnoty v hre, čo spôsobilo zmenu v pamäti. Tento krok sa musel opakovať niekoľkokrát, pričom každá zmena hodnoty v hre ma priviedla k menšiemu množstvu potenciálnych adries, kde sa hodnoty ukladali.
3. **Zmena hodnôt v pamäti:** Po viacerých skenoch a filtrovaní som našiel hodnoty, ktoré sa menili, keď som upravoval muníciu a životy. Tieto hodnoty som následne upravil priamo v Cheat Engine, čím som dosiahol efekt nekonečnej munície a životov.
4. **Zmena rýchlosti hry:** Okrem toho, že som mal možnosť manipulovať so základnými hernými hodnotami ako životy a munícia, pomocou Cheat Engine som dokázal upraviť aj rýchlosť hry. Táto funkcia je v Cheat Engine zabudovaná a umožňuje výrazne zmeniť rýchlosť vykonávania herného cyklu, čo môže viesť k zaujímavým zmenám v hernom zážitku.

Experiment bol úspešný, keďže som dokázal manipulovať s pamäťou hry a dosiahnuť požadované efekty. Po zmene hodnôt som mal plný prístup k meneniu herných údajov, čo umožnilo získať nekonečné zásoby munície a životov, ako aj upravovať rýchlosť hry. Tento experiment ukázal, ako môže nástroj ako Cheat Engine byť využitý na zmenu základných herných mechanizmov v offline prostredí.

Záver

V rámci tohto projektu som sa zameral na experimenty s detekciou cheatovania v online hrách, pričom sme testovali efektivitu rôznych anti-cheat systémov, ako sú Valve Anti-Cheat (VAC), Easy Anti-Cheat (EAC) a Vanguard. Testovanie prebiehalo na hrách ako Deadlock, CounterStrike, Valorant či Rust, ktoré sú vybavené pokročilými ochrannými mechanizmami. Výsledky experimentov ukázali, že tieto systémy dokážu účinne detegovať pokusy o manipuláciu s pamäťou a tradičné metódy cheatovania, vrátane wallhackov. Avšak mali problém s jednoduchšími cheatmi, ako je triggerbot. Tento cheat bol úspešný proti VAC, ale neprešiel detekciou v systéme Vanguard.

Všetky tieto experimenty sú zaznamenané na videách, ktoré sú súčasťou tejto práce, a poskytujú vizuálny dôkaz o interakcii medzi rôznymi cheatovacími technikami a anti-cheat ochranami.

Aj napriek tomu, že anti-cheat systémy sú čoraz silnejšie a sofistikovanejšie, vývojári cheatov neustále pracujú na obchádzaní týchto ochranných mechanizmov. Herný priemysel neustále napreduje, prinášajúc nové inovácie v oblasti ochrany proti podvodom, no zároveň aj nové metódy, ako tieto ochrany obísť. Preto je nevyhnutné, aby sa systémy detekcie podvodov neustále vyvíjali a prispôbovali novým hrozbám.

Tento projekt ukázal, že boj proti cheatovaniu je neustály proces, ktorý vyžaduje nielen technologický pokrok, ale aj flexibilitu a neustále monitorovanie. S ohľadom na rýchly vývoj v tejto oblasti je jasné, že zabezpečenie férového herného prostredia pre všetkých hráčov bude naďalej výzvou.

Referencie

CONSALVO, Mia. *Cheating: Gaining advantage in videogames*. mit press, 2009.

WEBB, Steven Daniel; SOH, Sieteng. Cheating in networked computer games: a review. In: *Proceedings of the 2nd international conference on Digital interactive media in entertainment and arts*. 2007. p. 105-112.

KÜCKLICH, Julian. Forbidden pleasures: Cheating in computer games. *The pleasures of computer gaming: Essays on cultural history, theory and aesthetics*, 2008, 52-71.

SUDHEER, Saqif Ayaan. Fortifying Game Security: Unveiling Vulnerabilities in Valve Anti Cheat and a Novel User Mode Anti-Cheat Solution. 2023.

LUKAS, Mihael; TOMICIC, Igor; BERNIK, Andrija. Anticheat system based on reinforcement learning agents in unity. *Information*, 2022, 13.4: 173.

<https://irdeto.com/blog/cheating-in-games-everything-you-always-wanted-to-know-about-it> 2.10.2024 22:00

<https://www.makeuseof.com/what-is-anti-cheat-software/> 2.10.2024 22:15

<https://www.thefastcode.com/cs-czk/article/what-is-easyanticheat-exe-and-why-is-it-on-my-computer> 2.10.2024 22:50

<https://www.itcomplet.sk/blog/typy-virusov> 2.10.2024 23:30

https://sk.wikipedia.org/wiki/Antivírusový_softvér 3.10.2024 1:00

<https://www.easy.ac/en-US> 3.10.2024 1:30

<https://www.ea.com/cs-cz/news/ea-anticheat-safety> 10.11.2024 21:00

https://en.wikipedia.org/wiki/Cheating_in_video_games 10.11.2024 21:40

https://en.wikipedia.org/wiki/Cheating_in_online_games 10.11.2024 22:15

<https://www.cyberark.com/resources/threat-research-blog/a-brief-history-of-game-cheating> 13.11.2024 22:00

https://wiki.cheatengine.org/index.php?title=Tutorials:Cheat_Engine_Tutorial_Guide_x64 13.11.2024 23:00

chatgpt.com – použitý pre zlepšený syntax a kontrola gramatiky

deepl.com – použitý pre preklad